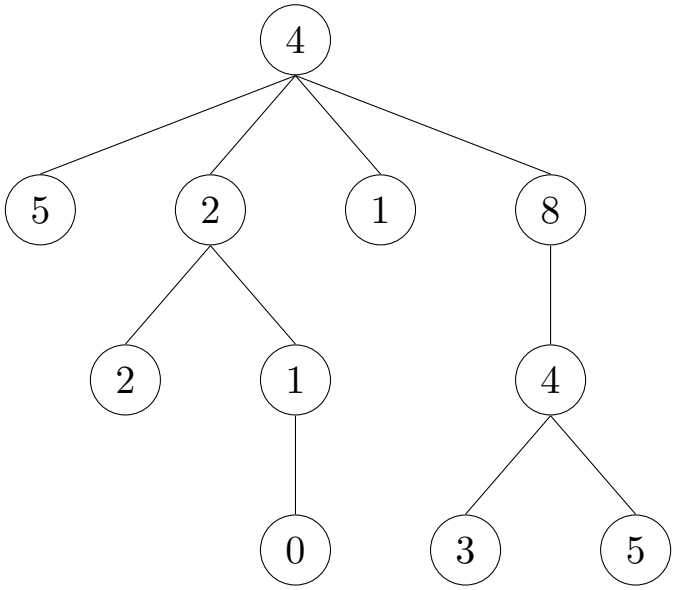
TREES, MUTABILITY AND ITERATORS Solutions

COMPUTER SCIENCE MENTORS 61A

October 6 – October 13, 2025

1 Trees

Trees are a kind of recursive data structure. Each tree has a **root label** (which is some value) and a sequence of **branches**. Trees are "recursive" because the branches of a tree are trees themselves! A typical tree might look something like this:



This tree's root label is 4, and it has 4 branches, each of which is a smaller tree. The 6 of the tree's **subtrees** are also **leaves**, which are trees that have no branches.

Trees may also be viewed **relationally**, as a network of nodes with parent-child relationships. Under this scheme, each circle in the tree diagram above is a node. Every non-root node has one parent above it and every non-leaf node has at least one child below it.

Trees are represented by an abstract data type with a `tree` constructor and `label` and `branches` selectors. The `tree` constructor takes in a label and a list of branches and returns a tree. Here's how one would construct the tree shown above with `tree`:

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

```
tree(4,
    [tree(5),
     tree(2,
        [tree(2),
         tree(1,
            [tree(0)])]),
     tree(1),
     tree(8,
        [tree(4,
            [tree(3), tree(5)])])])
```

The implementation of the ADT is provided here, but you shouldn't have to worry about this too much. (Remember the abstraction barrier!)

```
def tree(label, branches=[]):
        return [label] + list(branches)

def label(tree):
        return tree[0]

def branches(tree):
        return tree[1:] # returns a list of branches
```

Because trees are recursive data structures, recursion tends to a be a very natural way of solving problems that involve trees.

- The **recursive case** for tree problems often involves recursive calls on the branches of a tree.
- The **base case** is often reached when we hit a leaf because there are no more branches to recurse on.

1. Define the function `factor_tree` which returns a *factor tree*. Recall that in a factor tree, multiplying the leaves together is the prime factorization of the root, n.

```
def factor_tree(n):
    for i in _____________________:
        if _____________________________:
            return tree(_____, ___________________________________)
    ___________________________________

    for i in range(2, n):
        if n % i == 0:
            return tree(n, [factor_tree(i), factor_tree(n // i)])
    return tree(n)
```

2. Write the function `even_square_tree`, which takes in a tree `t` and returns a new tree with only the even labels squared.

```
def even_square_tree(t):
    '''
    >>> t = tree(2, [tree(1), tree(4)])
    >>> even_square_tree(t)
    tree(4, [tree(1), tree(16)])
    '''
    ____________________________________________

    if ____________________________________________:

        return ____________________________________

    else:

        return ____________________________________
```

```
def even_square_tree(t):
    branches_s = [even_square_tree(b) for b in branches(t)]
    if label(t) % 2 == 0:
        return tree(label(t) * label(t), branches_s)
    else:
        return tree(label(t), branches_s)
```

2 Mutability

1. Draw the environment diagram that results from running the following code.

```
pom = [16, 15, 13]
pompom = pom * 2
pompom.append(pom[:])
pom.extend(pompom)
```

https://goo.gl/ZU1V7h

2. Given some list `lst` of numbers, mutate `lst` to have the accumulated sum of all elements so far in the list. If `lst` is a deep list, mutate it to similarly reflect the accumulated sum of all elements so far in the nested list. Your function should return an integer representing your "accumulated" sum (sum of all numbers in your list). You may not need all lines provided.

Hint: The **`isinstance`** function returns True for **`isinstance`**`(l, list)` if `l` is a list and False otherwise.

```
def accumulate(lst):
    '''
    >>> l = [1, 5, 13, 4]
    >>> accumulate(l)
    23
    >>> l
    [1, 6, 19, 23]
    >>> deep_l = [3, 7, [2, 5, 6], 9]
    >>> accumulate(deep_l)
    32
    >>> deep_l
    [3, 10, [2, 7, 13], 32]
    '''
    sum_so_far = 0
    for ____________________________________________:

        ____________________________________________

        if isinstance(____________________, list):

            inside = ____________________________

            ____________________________________

        else:
            ____________________________________

            ____________________________________

    ________________________________________
```

```
def accumulate(lst):
    sum_so_far = 0
    for i in range(len(lst)):
        item = lst[i]
        if isinstance(item, list):
            inside = accumulate(item)
            sum_so_far += inside
        else:
            sum_so_far += item
            lst[i] = sum_so_far
    return sum_so_far
```

3 Iterators

1. What Would Python Display? If you think nothing will be displayed, write N/A.

```
>>> a = [1, 2, 3]
>>> x = iter(a)
```

```
N/A
```

```
>>> next(x)
```

```
1
```

```
>>> next(x)
```

```
2
```

```
>>> y = iter(a)
>>> next(y)
```

```
1
```

```
>>> next(x)
```

```
3
```

```
>>> next(x)
```

```
StopIteration Error
```

```
>>> z = iter(y)
>>> next(z)
```

```
2
```

```
>>> [next(y), next(y), next(z)]
```

```
[2, 3, 3]
```

```
>>> a = iter(filter(lambda x: x % 2, map(lambda x: x - 1, range(10))))
>>> next(a)
```

```
-1
```

```
>>> reduce(lambda x, y: x + y, a)
```

```
16
```

2. Write a generator that will take in two iterators and compares the first element of each iterator and yields the smaller of the two values. For simplicity, ignore any StopIteration that might result when we run out of values to yield.

```
def interleave(iter1, iter2):
    '''
    >>> gen = interleave(iter([1, 3, 5, 7, 9]),
                         iter([2, 4, 6, 8, 10]))
    >>> for elem in gen:
    ...     print(elem)
    1
    2
    3
    4
    5
    6
    7
    8
    9
    '''


    t1, t2 = next(iter1), next(iter2)
    while True:
        if t1 > t2:
            yield t2
            t2 = next(iter2)
        else:
            yield t1
            t1 = next(iter1)
```